

5일 차

다중 분류 모델링

이번 차시에서 주목할 키워드

- 다중 분류
- 소프트맥스 함수
- CrossEntropyLoss
- 정수 인코딩 레이블 인코딩

5일 차 학습 목표

- 다중 분류의 개념을 이해하고 이진 분류와의 차이를 설명할 수 있습니다.
- 다중 분류 모델의 출력층 구조, 활성화 함수, 손실 함수의 역할을 이해합니다.
- 범주형 목표값을 정수 인코딩하고 다중 분류 학습을 위한 데이터 전처리를 직접 구성할 수 있습니다.

4일 차에서는 두 가지 범주 중 하나를 예측하는 이진 분류 모델을 배웠습니다. 이번 차에서는 한 단계 더 확장된 다중 분류(Multi-Class Classification) 문제를 다룹니다. 다중 분류는 세 개 이상의 범주 중 하나를 분류하는 문제입니다. 예를 들어 손글씨로 쓴 숫자 이미지를 보고 0부터 9 사이의 숫자 중 하나를 예측하는 MNIST 문제처럼 우리가 실생활에서 마주하는 다양한 분류 과제는 주로 다중 분류 문제가 많습니다. 다중 분류 모델의 출력층 구성과 활성화 함수, 손실 함수가 이진 분류와 어떻게 다른지 차근차근 살펴보고 실습으로 직접 구현하겠습니다.

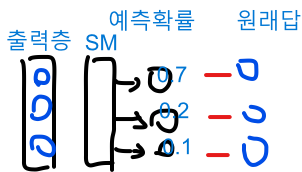
다중 분류 모델 개념 이해

다중 분류(Multi-Class Classification)는 목표값(target)의 범주가 3개 이상인 경우입니다. 이진 분류에서는 출력층에 시그모이드(Sigmoid) 활성화 함수를 적용해 0~1 사이의 확률값을 예측했습니다. 범주가 2개이므로 하나의 확률값을 이용해 0이나 1로 나누면 되었고 따라서 출력층 노드도 1개면 충분했습니다. 그러나 다중 분류 문제는 고려해야 할 범주가 여러 개이므로 출력 노드 하나로는 문제를 해결할 수 없습니다. 예컨대 특정 이미지를 보고 고양이, 강아지, 토끼 중 어떤 이미지인지 구분하는 문제라면 하나의 확률값으로 셋 중 하나를 분류할 수는 없습니다. 그렇다면 다중 분류 모델은 출력층을 어떻게 구성할까요? 지금부터 차근차근 설명합니다.

다중 분류 모델의 출력층

이진 분류 모델 vs. 다중 분류 모델

이진 분류와 다중 분류의 가장 큰 차이는 출력층의 구조와 활성화 함수입니다. 즉, 이진 분류는 하나의 노드에서 확률을 출력하는 반면, 다중 분류는 범주마다 하나의 출력 노드를 두고 각각의 확률을 계산합니다. 그리고 각각의 확률들의 합은 1입니다.



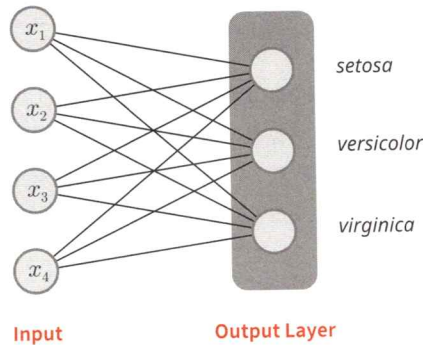
예측값 3개 나오는데 원래답은 1개(0,1,2 중 하나)
so 원핫으로 답 3개로 만들어줘야. ([1,0,0],[0,1,0] 등)

[표 5-1] 이진 분류, 다중 분류의 구조 비교

분류 유형	출력층 구조	활성화 함수	출력값의 의미
이진 분류	출력 노드 1개	시그모이드	0~1 사이의 확률
다중 분류	출력 노드는 범주의 개수만큼	소프트맥스	각 범주의 확률



다음 절에서 예제로 사용할 붓꽃 데이터(이하 iris 데이터)는 목꽃값 범주가 *setosa*, *versicolor*, *virginica* 3개로 구성된 데이터 세트입니다. 이 데이터를 다중 분류 모델로 구성할 때는 출력 노드도 3개가 있어야 합니다. 각 노드의 예측 결과는 각 범주의 확률값입니다. 예를 들어 어떤 입력값으로 세 범주의 확률을 예측한다고 합시다. 예측 결과가 *setosa*일 확률은 0.6, *versicolor*는 0.3, *virginica*는 0.1과 같다면 세 범주 확률값($0.6 + 0.3 + 0.1$)의 합은 1입니다.



[그림 5-1] Iris 데이터 세트의 다중 분류 모델 구조

원래답(label) 목꽃값에 대한 전처리

딥러닝으로 다중 분류 모델을 만들려면 입력 데이터뿐만 아니라 목꽃값(target) 데이터도 적절한 형태로 준비해야 합니다. 특히 분류 문제는 목꽃값이 *setosa*, *versicolor*, *virginica*와 같은 문자열로 되어 있는 경우가 많으므로 모델이 이해할 수 있는 숫자 형태로 변환해야 합니다.

이진 분류 모델에서는 목꽃값이 0과 1로 변환되어야 했지만, 다중 분류 모델에서는 정수형 인덱스(0, 1, 2, ...)를 제공해야 합니다. 이 정수는 출력층 소프트맥스 함수의 결과가 어떤 인덱스에 해당하는지를 의미합니다. iris 데이터처럼 범주가 총 3개(*setosa*, *versicolor*, *virginica*)라면 각각의 범주값을 [0, 1, 2]로 변경하면 됩니다. 한 가지 주의할 점은 반드시 0부터 시작해 1씩 증가하는 값으로 변환(정수 인코딩)해야 합니다.

다중 분류에서는 목푫값을 원-핫 인코딩(One-Hot Encoding)하는 것으로 알고 있는 분들이 많습니다. 하지만 파이토치에서는 그렇지 않습니다. 곧 배우게 될 다중 분류 모델의 손실 함수인 교차 엔트로피(`nn.CrossEntropyLoss`)는 내부적으로 소프트맥스 함수를 적용한 다음, 정수 인코딩된 목푫값(target)과 비교해 손실을 계산합니다. 따라서 목푫값은 정수 인코딩된 1차원 데이터를 그대로 사용합니다.



원-핫 인코딩(One-Hot Encoding), 정수 인코딩(Integer Encoding)?

데이터 전처리 과정에서 자주 등장하는 단어가 인코딩(Encoding)입니다. 그리고 인코딩의 짝인 단어가 디코딩(Decoding)입니다. 인코딩은 기존 정보를 어떤 목적에 맞게 변형하는 작업이고 디코딩은 변형된 정보를 다시 원래대로 되돌리는 작업입니다. 그렇다면 원-핫 인코딩과 정수 인코딩은 어떤 목적을 가지고 정보를 변형하는 것일까요?

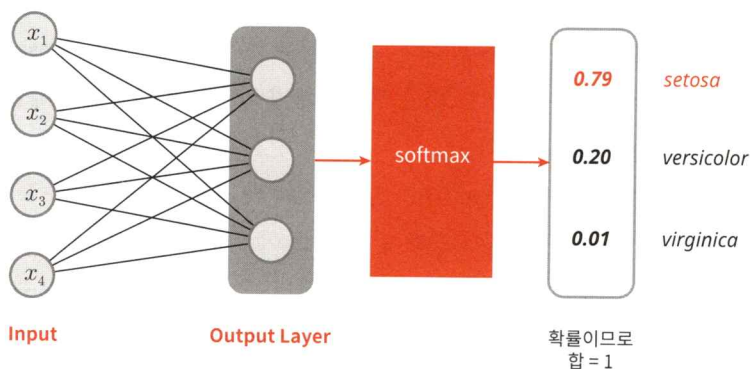
정수 인코딩은 각각의 범주에 고유한 정수를 부여하는 방식입니다. 예를 들어 세 개의 범주가 있을 때 *setosa*는 0, *versicolor*는 1, *virginica*는 2로 표현합니다. 간단하고 직관적인 방식이지만, 숫자 간의 크기 차이가 모델에서 의미 있는 순서나 거리로 잘못 해석될 수 있다는 단점도 있습니다.

원-핫 인코딩은 각각의 범주를 벡터로 표현하는데, 해당 범주에만 1을 주고 나머지는 모두 0으로 설정하는 인코딩 방식입니다. 예를 들어 *versicolor*는 정수 인코딩된 값이 1이므로 [0, 1, 0]으로 표현합니다. 이렇게 하면 범주 간의 거리나 순서에 대한 잘못된 정보를 제거할 수 있는 장점이 있습니다. 원-핫 인코딩을 하려면 반드시 정수 인코딩을 먼저 해야 하고 0부터 시작해야 합니다.

Cross Entropy에서 원핫 해줌. so, 우리가 해줄 필요는 없지만 자체적으로는 필요함.

출력층의 동작 방식

[그림 5-2]에 표현되어 있는 출력층의 동작을 단계별로 설명하면 다음과 같습니다.



[그림 5-2] 출력층 결과에 소프트맥스 연산을 적용해 확률값으로 변환

1. 데이터(x_1, x_2, x_3, x_4)를 모델에 입력합니다.
2. 가중치를 적용해 세 개의 출력 노드에서 점수를 생성(계산)합니다. 이때 생성되는 점수는 확률값이 아닌 수치값입니다. 4일 차 이진 분류 모델에서 확률로 변환되기 전 숫자를 로짓(Logit)이라고 했습니다.
3. 로짓값은 소프트맥스 함수에 입력되어 각 범주의 확률값으로 변환됩니다. 소프트맥스 함수는 여러 로짓값을 묶어 확률값으로 변환하는 함수입니다. 이진 분류 모델의 시그모이드처럼 다중 분류 모델 출력층의 활성화 함수라고 이해하면 됩니다.
4. 이 중에서 가장 높은 확률을 가진 범주가 최종 예측값이 됩니다.

[그림 5-2]에서는 *setosa*일 확률이 0.79, *versicolor*는 0.20, *virginica*는 0.01이므로 최종 예측 범주는 *setosa*입니다.

소프트맥스 함수

이진 분류에서 사용하는 시그모이드(Sigmoid) 함수는 출력층의 노드가 1개이고 이 노드에서 계산한 하나의 로짓값을 0~1 사이의 확률로 출력합니다. 하지만 다중 분류에서는 분류해야 할 범주의 수만큼 노드를 출력층으로 두고 각각의 노드에서 나오는 로짓값으로 각각의 확률값을 만듭니다. 이때 사용하는 함수가 바로 소프트맥스입니다. 이 함수의 식은 다음과 같습니다.

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum e^{x_i}}$$

예를 들어 출력층의 3개 노드에서 계산된 점수(로짓값)가 2.5, 1.2, -0.5 라고 할 때 이를 수식에 대입해 계산하면 다음과 같습니다.

- 첫 번째 노드(*setosa*)의 확률값: $\frac{e^{2.5}}{e^{2.5} + e^{1.2} + e^{-0.5}} = 0.79$
- 두 번째 노드(*versicolor*)의 확률값: $\frac{e^{1.2}}{e^{2.5} + e^{1.2} + e^{-0.5}} = 0.20$
- 세 번째 노드(*virginica*)의 확률값: $\frac{e^{-0.5}}{e^{2.5} + e^{1.2} + e^{-0.5}} = 0.01$

세 확률값의 합($0.79 + 0.20 + 0.01$)은 1이고 여기서 확률이 가장 높은 *setosa*(0.79) 범주가 최종 예측 결과가 됩니다.

정리하면 이진 분류와 다중 분류의 가장 큰 차이는 출력층의 노드 개수와 활성화

함수입니다. 이진 분류는 시그모이드 함수로 단일 확률값을 출력하지만, 다중 분류는 소프트맥스 함수를 사용해 각 노드의 출력값을 확률값으로 변환합니다.

다중 분류 모델의 손실 함수 이해

분류 문제에서는 예측이 얼마나 목푫값과 일치하는지 평가하기 위해 교차 엔트로피(Cross Entropy)라는 손실 함수를 사용합니다. 이 함수는 예측한 확률값이 목푫값과 얼마나 가까운지 계산하며 값이 작을수록(가까울수록) 더 정확한 예측임을 의미합니다.

이진 분류에서는 두 가지 범주 중 하나를 예측하므로 모델의 출력은 하나의 숫자입니다. 여기에 시그모이드 함수를 적용해 0과 1 사이의 확률로 변환합니다. 이후 이 확률과 실제 목푫값(0 또는 1)을 비교해 이진 분류 교차 엔트로피(Binary Cross Entropy) 손실을 계산합니다.

반면 다중 분류는 예측해야 할 범주가 3개 이상으로 늘어나기 때문에 출력층도 범주의 수만큼 노드가 필요합니다. 출력값에 소프트맥스(Softmax) 함수를 적용하면 각각의 범주에 속할 확률이 계산됩니다. 그리고 소프트맥스 함수로 얻은 확률과 실제 목푫값을 비교해 다중 범주용 교차 엔트로피 손실을 계산합니다.

이제 다중 분류 상황에서 사용되는 교차 엔트로피 손실 함수의 정의와 계산 방식을 살펴보고 이진 분류와 어떤 차이가 있는지 설명하겠습니다.

교차 엔트로피 공식

다중 분류를 위한 교차 엔트로피(Cross Entropy) 식은 4일 차에서 설명한 이진 교차 엔트로피 식과 비슷합니다. 다음 수식에 대해 차근차근 설명하겠습니다.

$$\text{Cross Entropy} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log p_{i,c}$$

- N : 샘플 수(배치 크기) 확률(0~1)을 오차(0~+inf)로 바꿔준 것.
- C : 범주 수
- i : 데이터의 인덱스
- $y_{i,c}$: 실제 목푫값(Label)
- $p_{i,c}$: 예측 확률(소프트맥스를 거친 모델의 출력값)

공식을 하나씩 뜯어 보면 다음과 같습니다.

- ① $\frac{1}{N} \sum_{i=1}^N$: 이런 모양의 수식이 나오면 그냥 ‘평균’으로 읽으라고 했지요? i 는 데이터 세트의 행 인덱스입니다. 데이터 세트에 데이터가 총 N 개 있으며 각각의 데이터에서 어떤 값의 평균을 계산하라는 의미입니다.
- ② $-\log p_{i,c}$: 예측 확률($p_{i,c}$)에 로그를 취했습니다. 식 맨 앞에 -(마이너스)가 붙어 있는데, $-\log$ 에서 -만 맨 앞으로 보낸 겁니다. $-\log$ 는 핵심적인 수식으로 교차 엔트로피에서 오차를 계산합니다. 다음 쪽 [그림 5-3]의 오른쪽 그래프가 바로 $-\log$ 입니다. 4일 차에서 설명했던 내용을 떠올리길 바랍니다. 여기서 $p_{i,c}$ 는 i 행에서 목פות값 범주 c 의 확률입니다. 앞의 예제에서는 *setosa*일 확률이 0.79, *versicolor*는 0.20, *virginica*는 0.01이었습니다.
- ③ $y_{i,c} \log p_{i,c}$: ②의 수식에 $y_{i,c}$ 를 곱합니다. 목פות값이 *setosa*라면 *setosa*만 1이고 *versicolor*와 *virginica*는 0입니다.
- ④ $\sum_{c=1}^C y_{i,c} \log p_{i,c}$: 목פות값 범주별로 ③을 계산해 모두 더하는 부분입니다. 앞의 예제로 이를 계산하면 다음과 같습니다.

$$1 \times 0.79 + 0 \times 0.20 + 0 \times 0.01 = 0.79$$

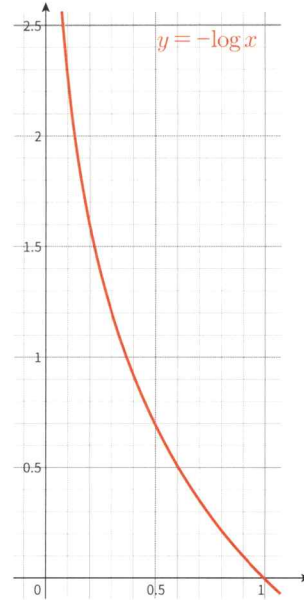
이해가 되시나요? 조금 어려울 것 같아 그림을 보며 다시 한번 설명하겠습니다.

교차 엔트로피 계산 예제

이해를 돕고자 [그림 5-3]의 예제로 다시 설명합니다. [그림 5-3]의 오른쪽 그래프는 식이 $y = -\log x$ 입니다. 4일 차에서 이진 교차 엔트로피를 설명하면서 이미 소개했습니다. [그림 5-3]에서 왼쪽 표를 보면 목פות값(y)은 0, 1, 2로 정수 인코딩되어 있고, 예측값($\hat{y}$)은 세 범주의 확률값입니다. 그렇다면 이 두 값 사이의 오차는 어떻게 계산할까요?

왼쪽 표에서 첫 번째 행을 보면 목פות값(y)은 1, 예측값($\hat{y}$)은 [0.1, 0.6, 0.3]입니다. 목פות값 1을 인덱스로 사용해 예측값의 확률들 중 인덱스 1의 확률값을 뽑습니다. 여기서는 0.6입니다. 이 값을 $-\log$ 로 계산해 오차를 구하면 0.51입니다. 두 번째 행도 계산하면 목פות값(y)은 0, 예측값($\hat{y}$)은 [0.3, 0.2, 0.5]입니다. 목פות값 0을 인덱스로 사용한다고 했지요? 예측값에서 0번 인덱스 위치의 확률값은 0.3입니다. 이 확률값으로 $-\log$ 계산을 하면 1.20입니다. 첫 번째 행은 범주 1에 대한 확률을 0.6으로 예측했고, 두 번째 행은 범주 0에 대한 확률을 0.3으로 예측했습니다. 첫 번째 행은 목פות값 범주에 0.6이라는 꽤 높은 확률을 줬고, 두 번째 행은 목פות값 범주에

y	$\hat{y}$	err_1
1	[0.1, 0.6, 0.3]	$-\log(0.6) = 0.51$
0	[0.3, 0.2, 0.5]	$-\log(0.3) = 1.20$
2	[0.2, 0.4, 0.4]	$-\log(0.4) = 0.91$
1	[0.1, 0.9, 0.0]	$-\log(0.9) = 0.10$
0	[0.7, 0.1, 0.2]	$-\log(0.7) = 0.36$
0	[0.5, 0.3, 0.2]	$-\log(0.5) = 0.69$
2	[0.1, 0.1, 0.8]	$-\log(0.8) = 0.22$



[그림 5-3] 예측 결과를 교차 엔트로피로 계산하는 과정

0.3이라는 낮은 확률을 줬습니다. 그래서 첫 번째 행의 예측이 더 잘 맞았다고 볼 수 있고 그만큼 오차도 더 적게 나옵니다($0.51 < 1.20$).

이런 식으로 각 행의 손실값을 모두 계산한 뒤 평균을 구하면 그 값이 바로 다중 분류에서 사용하는 교차 엔트로피(`nn.CrossEntropyLoss`)입니다.

이 함수를 사용할 때 주의할 점이 있습니다. 바로 교차 엔트로피 함수 내부에 소프트맥스 연산이 포함되어 있다는 점입니다. 그러므로 모델을 설계할 때는 출력층에 소프트맥스 함수를 추가할 필요가 없습니다. 다만 학습을 마친 다음, 예측할 때는 예측 결과가 그냥 로짓값입니다. 이 값을 소프트맥스 함수로 변환해야 범주별로 확률값을 구할 수 있습니다. 이 변환에 대해서는 예제 코드를 작성하면서 다시 한번 설명합니다.

다중 분류 모델링 실습 1 - iris

이번 절에서는 다중 분류 모델링(Multi-Class Classification)을 실습합니다. 다중 분류 모델링을 위해 붓꽃(iris) 품종 분류 데이터를 사용합니다. 이 데이터 세트는 머신러닝/딥러닝 입문자들이 분류 문제를 배울 때 가장 널리 사용하는 대표적인 예제 데이터입니다. 이 데이터는 유명한 통계학자 로널드 피셔(Ronald Fisher)가 처음

소개했는데, 세 종류의 붓꽃 품종을 꽃잎과 꽃받침의 길이와 너비를 이용해 분류하는 문제입니다.

데이터 세트의 변수는 다음과 같습니다.

- Sepal.Length: 꽃받침 길이
- Sepal.Width: 꽃받침 폭
- Petal.Length: 꽃잎 길이
- Petal.Width: 꽃잎 폭
- Species: 품종(setosa, versicolor, virginica)

이제 환경 준비부터 모델링 및 모델 평가까지 다중 분류 모델링 절차와 방법을 코드로 구현합니다. 전과 동일하게 코랩에서 새 파일을 생성하고 파일 이름은 '5일차_다중분류모델실습.ipynb'로 명명합니다. 나머지는 이전 차시와 거의 동일합니다.

환경 준비

라이브러리 로딩, 디바이스 준비 및 데이터 로딩

이전과 동일하게 기본 라이브러리, 데이터 전처리를 위한 `sklearn`, 모델링을 위한 `torch` 라이브러리를 불러오고 디바이스도 준비합니다.

CODE

```
# 라이브러리 로딩
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import *
from sklearn.preprocessing import MinMaxScaler
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
from torch.optim import Adam

# 디바이스 준비
device = "cuda" if torch.cuda.is_available() else "cpu"
```

라이브러리와 디바이스를 준비했습니다. 기본 설정 코드는 여러 프로젝트에서 반복적으로 사용되므로 별도로 잘 정리해 두었다가 필요할 때마다 불러와서 사용하는 것이 효과적입니다.

사용자 정의 함수 생성

데이터 전처리, 모델링의 주요 작업들을 자동화하기 위한 함수를 생성합니다. 이전 모델링 실습에서 모두 다루었던 함수들입니다. 학습(train), 검증(evaluate), 학습 곡선(dl_learning_curve) 함수는 이전 차시와 동일하게 사용합니다. 다만 데이터 로더를 만드는 make_DataSet 함수는 입력값과 목표값을 처리하는 부분에서 약간 다릅니다.

CODE

```
## 학습 함수
def train(dataloader, model, loss_fn, optimizer, device):
    size = len(dataloader.dataset) # 전체 데이터 세트의 크기
    num_batches = len(dataloader) # 배치 크기
    tr_loss = 0

    model.train() # 학습 모드로 설정
    for x, y in dataloader: # batch 단위 로딩
        x, y = x.to(device), y.to(device) # 디바이스 지정

        # Feed Forward(오차 순전파)
        pred = model(x)
        loss = loss_fn(pred, y)
        tr_loss += loss

        # Backpropagation(오차 역전파)
        loss.backward() # 역전파를 통해 각 파라미터의 손실 기울기 계산
        optimizer.step() # 옵티마이저가 모델의 파라미터를 업데이트
        optimizer.zero_grad() # 옵티마이저의 기울기값 초기화
    tr_loss /= num_batches # 모든 배치에서의 오차 평균
    return tr_loss.item()

## 검증 평가 함수
def evaluate(x_val_tensor, y_val_tensor, model, loss_fn, device):
    model.eval() # 모델을 평가 모드로 설정
    with torch.no_grad(): # 평가 과정에서 기울기를 계산하지 않도록 설정
        x, y = x_val_tensor.to(device), y_val_tensor.to(device)
        pred = model(x)
        eval_loss = loss_fn(pred, y).item() # 예측값 pred와 목표값 y 사이의 오차 계산
    return eval_loss, pred

## 학습 곡선 함수
def dl_learning_curve(tr_loss_list, val_loss_list):
    epochs = list(range(1, len(tr_loss_list)+1))
    plt.plot(epochs, tr_loss_list, label='train_err', marker = '.')
    plt.plot(epochs, val_loss_list, label='val_err', marker = '.')
    plt.ylabel('Loss')
```

```
plt.xlabel('Epoch')
plt.legend()
plt.grid()
plt.show()
```

세 함수를 먼저 생성했습니다. 이 함수들 중 `train` 함수의 내부 코드들은 딥러닝 모델링의 학습 절차를 이해하는 데 중요하므로 다시 한번 코드를 꼼꼼히 읽어 보기 바랍니다.

다중 분류 모델을 위한 데이터 로더 함수

데이터 로더를 생성하는 `make_DataSet` 함수는 3~4일 차에서 이미 설명했습니다. 그런데 이번 차시에서 이 함수의 내용이 조금 달라집니다. 먼저 코드를 작성합니다.

CODE

```
## 데이터 로더 선언 함수
def make_DataSet(x_train, x_val, y_train, y_val, batch_size = 32) :

    # 텐서로 변환
    x_train_tensor = torch.tensor(x_train, dtype=torch.float32)
    y_train_tensor = torch.tensor(y_train, dtype=torch.long)
    x_val_tensor = torch.tensor(x_val, dtype=torch.float32)
    y_val_tensor = torch.tensor(y_val, dtype=torch.long)

    # TensorDataset 생성: 텐서 데이터 세트로 합치기
    train_dataset = TensorDataset(x_train_tensor, y_train_tensor)

    # DataLoader 생성
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle = True)
    return train_loader, x_val_tensor, y_val_tensor
```

함수의 내부 절차는 동일합니다. `make_DataSet` 함수는 `x_train`, `x_val`, `y_train`, `y_val`, `batch_size`를 입력으로 받습니다. 그리고 입력 데이터는 모두 텐서로 변환합니다. 그중 학습 데이터는 텐서 데이터 세트로 먼저 변환하며 이 데이터 세트를 이용해 데이터 로더를 생성합니다. 결과로 `make_DataSet` 함수는 데이터 로더와 검증 텐서 데이터 세트를 반환합니다.

그런데 앞에서 만들었던 `make_DataSet` 함수와 비교하면 두 곳이 다릅니다. 바로 `y_train_tensor`와 `y_val_tensor`를 만드는 부분입니다.

이진 함수와 비교해 어떤 차이점이 있는지 회귀, 이진 분류, 다중 분류 코드를 비교하면 다음과 같습니다. 비교하는 내용이므로 코드를 작성할 필요는 없습니다.

CODE

```
# 회귀
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1) ①

# 이진 분류
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)

# 다중 분류
y_train_tensor = torch.tensor(y_train, dtype=torch.long) ②
```

① y_train은 1차원 넘파이 배열입니다. 이 배열을 실수 타입(float32)의 텐서로 만들고 view 메서드를 이용해 2차원으로 변환합니다.

② 1차원 넘파이 배열(y_train.values)을 1차원 텐서로 만듭니다. 이때 데이터 타입은 정수(long)입니다.

왜 각각의 문제 유형에 따라 데이터의 형식과 구조 간에 차이가 생길까요?

모델에 여러 행의 데이터를 입력하면 예측 결과도 행(row)마다 하나씩 출력되어 일반적으로 2차원 텐서 형태가 됩니다. 이 예측값(2차원 텐서 구조)과 목표값을 비교해 오차를 계산합니다. 이때 목표값은 손실 함수가 기대하는 구조와 타입으로 준비되어야 합니다. 따라서 사용하는 손실 함수에 따라 목표값의 데이터 형식도 달라져야 합니다.

- 회귀 모델: 손실 함수 `MSELoss`는 두 실수 간의 제곱 오차를 계산하고 평균을 구합니다. 회귀 모델은 연속된 숫자를 예측하므로 목표값이나 예측값 모두 실수 타입입니다. 그리고 두 텐서의 비교 연산을 위해서는 구조가 같아야 하므로 목표값은 2차원 실수 타입의 텐서로 준비해야 합니다.
- 이진 분류 모델: `BCELoss`는 목표값과 $-\log(\text{확률값})$ 를 곱해 계산합니다. 두 텐서 간의 연산이므로 역시 2차원 구조의 실수 타입으로 지정해야 합니다.
- 다중 분류 모델: `CrossEntropyLoss`는 목표값을 클래스 인덱스(정수)로 사용합니다. 모델은 각 범주에 대한 확률 벡터를 출력하며, 손실 함수는 이 벡터와 목표값을 비교해 오차를 계산합니다. 그런데 인덱스는 정수 타입이므로 `CrossEntropyLoss` 내부에서는 목표값을 1차원 텐서로 받아 오차를 계산해야 합니다. 따라서 다중 분류 모델에서는 목표값을 1차원 정수 타입의 텐서로 만듭니다.

설명이 좀 길어졌습니다. 지금까지의 내용을 표로 정리하면 다음과 같습니다.

[표 5-2] 문제 유형별 y의 구조 비교

구분	목표값(y) 텐서의 차원	목표값(y) 데이터 타입	손실 함수
회귀	2	실수(float32)	MSELoss
이진 분류	2	실수(float32)	BCELoss
다중 분류	1	정수(long)	CrossEntropyLoss

문제 유형(회귀, 이진 분류, 다중 분류)에 따라 목표값 데이터의 구조와 자료형이 달라지는 이유를 살펴보았습니다. 조금 어려운 내용입니다. 하지만 여기서 기억해야 할 중요한 점은 손실 함수의 입력 형식에 맞게 목표값을 정확히 준비해야 오류 없이 학습이 진행된다는 것입니다.

데이터 전처리

데이터 전처리는 데이터 로딩, 데이터 세트 구성, y에 대한 전처리, 데이터 분할(train, val), 스케일링, 데이터 로더 준비로 진행합니다. 대부분 이전 차시에서 설명한 내용과 동일하지만, y를 정수로 인코딩하는 단계가 하나 더 추가되었습니다.

데이터 로딩 및 x, y 분할

앞에서 소개했던 iris 데이터 세트를 불러옵니다.

CODE

```
path = "https://bit.ly/irisdata_csv"
data = pd.read_csv(path)
data.head()
```

OUTPUT

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

코드 실행 결과를 보면 iris 데이터는 4개의 입력값(Sepal.Length, Sepal.Width, Petal.Length, Petal.Width)과 1개의 목표값(Species)으로 구성되어 있습니다.

이 데이터 세트를 x와 y로 분할합니다.

CODE

```
target = 'Species'
x = data.drop(target, axis = 1)
y = data.loc[:, target]
```

목표값(Species)을 중심으로 데이터를 x, y로 분할했습니다. 다음으로 y의 전처리를 설명합니다.

y의 전처리: 정수 인코딩

이번 다중 분류 모델의 y(target)는 setosa, versicolor, virginica로서 텍스트 형식의 범주값입니다. 앞에서 손실 함수 CrossEntropyLoss를 소개하면서 이 손실 함수에 입력할 목표값(y)은 1차원 정수 타입의 텐서이어야 한다고 했습니다. 그래서 setosa, versicolor, virginica를 정수 타입으로 바꾸는 정수 인코딩(Integer Encoding)을 수행합니다.

정수 인코딩을 처리하는 다양한 방법이 있지만, 여기서는 sklearn의 LabelEncoder 함수를 이용하겠습니다.

CODE

```
from sklearn.preprocessing import LabelEncoder ①
le = LabelEncoder() ②
y = le.fit_transform(y) ③
print(type(y))
```

OUTPUT

```
<class 'numpy.ndarray'>
```

- ① 정수로 인코딩하는 함수는 다양합니다. 여기서는 sklearn에서 제공하는 LabelEncoder 함수를 사용합니다.
- ② LabelEncoder 객체를 선언하고 변수 le에 저장합니다.
- ③ fit_transform 메서드에 y를 넣고 변환하면 정수 인코딩을 수행합니다.

코드 실행 결과를 보면 인코딩해 저장한 y는 넘파이 배열입니다. 따라서 make_DateSet 함수에 넣기 전에 y_train이나 y_val을 다시 넘파이 배열로 변환할 필요는 없습니다.



범주형 정보를 정수로 인코딩한 다음 원래대로 되돌리는 방법

범주값을 정수 인코딩하면 범주의 순서대로 0에서 시작해 1, 2, 3, ...순으로 증가하는 정숫값이 됩니다. 그런데 인코딩된 정숫값을 사용하다 보면 각 정수의 의미를 확인해야 할 경우가 생깁니다. 이때 다음 코드를 활용하면 됩니다.

CODE

```
print(le.classes_) ①  
y_orig = le.inverse_transform(y) ②
```

- ① classes_ 속성은 인코딩하기 전 범주의 목록을 보여 줍니다.
- ② inverse_transform 메서드를 사용하면 인코딩하기 전의 데이터로 변환할 수 있습니다.

train, val 분할, 스케일링, 데이터 로더 준비

데이터 전처리의 나머지 작업을 순차적으로 수행합니다. 이 부분 역시 이전과 동일합니다.

CODE

```
# train, val 분할  
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size=.3, random_state=20)  
  
# 스케일링  
scaler = MinMaxScaler()  
x_train = scaler.fit_transform(x_train)  
x_val = scaler.transform(x_val)  
  
# 데이터 로더  
train_loader, x_val_ts, y_val_ts = make_DataSet(x_train, x_val, y_train, y_val, 32)
```

환경 준비부터 데이터 준비까지 모두 마쳤습니다. 데이터 전처리 과정에서 회귀 및 이진 분류 모델링과 달라진 점을 다시 정리하면 다음과 같습니다.

- make_DataSet 함수에서 y 텐서의 구조는 1차원, 데이터 타입은 정수(long)로 지정
- y의 정수 인코딩이 필요

이 두 부분이 달라졌습니다. 다중 분류 모델링을 위한 손실 함수 CrossEntropyLoss

의 내부 계산 로직에 맞추기 위해 이 작업은 꼭 필요합니다. 이제 다중 분류 모델링으로 넘어갑니다.

모델링

모델 구조 선언

다중 분류 모델의 구조를 설계할 때도 회귀나 이진 분류 모델과 다른 점이 두 가지 있습니다.

- 출력층의 노드(Output Node) 수는 분류하려는 범주 수와 같다.
- 출력층 이후에 소프트맥스 함수가 필요하지만, 손실 함수(CrossEntropyLoss)에 소프트맥스 연산이 포함되므로 모델 구조를 설계하는 코드에는 소프트맥스를 포함하지 않는다.

이 두 가지를 기억하면서 층 하나로 이루어진 다중 분류 모델 구조를 다음과 같이 구현합니다.

CODE

```
n_feature = x.shape[1] ①
n_class = len(le.classes_) ②

# 모델 구조 설계
model = nn.Sequential(
    nn.Linear(n_feature, n_class), ③
).to(device)
```

① 입력변수(x, 특징)의 수를 변수 n_feature에 저장합니다.

② le.classes_ 속성에는 정수 인코딩하기 전의 범주 목록이 있습니다. len 함수로 범주 목록의 개수를 셉니다. 즉, 다중 분류 모델에서 범주의 수를 셉니다.

③ nn.Linear 함수가 하나이므로 출력층만 있는 구조입니다. feature 수만큼 입력으로 받고 target의 범주 수(n_class)만큼 출력합니다.

다중 분류 모델에서는 출력층의 노드 수가 범주 수와 같습니다. 또한 다중 분류 모델의 출력층에는 소프트맥스 함수가 추가되어야 하지만, 손실 함수(CrossEntropy Loss)에 소프트맥스 연산이 포함되므로 모델 구조를 작성할 때는 이를 생략합니다.

손실 함수와 옵티마이저

다음으로 손실 함수와 옵티마이저를 선언합니다. 이진 분류나 회귀 모델뿐만 아니라 다중 분류 모델에서도 옵티마이저는 Adam을 사용합니다. 하지만 손실 함수는 다중 분류 모델에서 사용하는 교차 엔트로피(`nn.CrossEntropyLoss`)를 사용합니다.

CODE

```
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=0.1)
```

학습 및 학습 곡선

이제 모델을 학습시킵니다. 학습과 학습 곡선은 회귀, 이진 분류 모델과 동일합니다.

CODE

```
epochs = 50
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model, loss_fn, optimizer, device)
    val_loss, _ = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)
    tr_loss_list.append(tr_loss)
    val_loss_list.append(val_loss)
    print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")
```

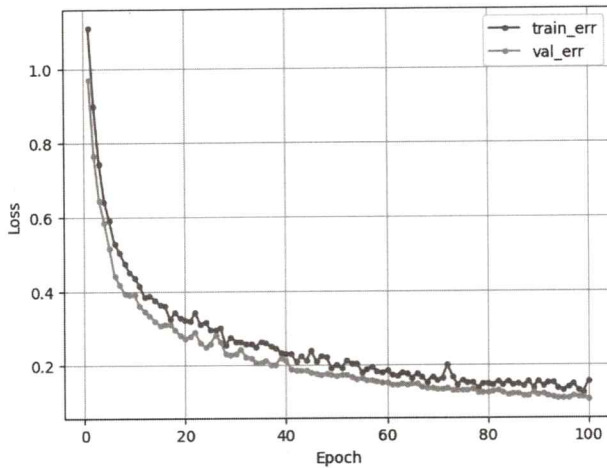
OUTPUT

```
Epoch 1, train loss : 1.110151, val loss : 0.968307
Epoch 2, train loss : 0.899381, val loss : 0.763941
Epoch 3, train loss : 0.743901, val loss : 0.643649
...
Epoch 98, train loss : 0.131630, val loss : 0.112698
Epoch 99, train loss : 0.126547, val loss : 0.112153
Epoch 100, train loss : 0.154866, val loss : 0.107857
```

학습 곡선으로 학습이 적절히 수행되었는지 살펴봅니다. 필자가 강의할 때 보면 학습 곡선 그래프를 어떻게 해석해야 할지 난감해 하는 분들이 많습니다. 학습 곡선은 학습이 적절히 진행되었는지 살펴볼 수 있는 매우 중요한 근거이므로 몇 번 더 반복해 설명합니다.

CODE

```
dl_learning_curve(tr_loss_list, val_loss_list)
```



[그림 5-4] 다중 분류 모델의 학습 곡선

[그림 5-4]의 학습 곡선을 보면 에포크가 증가할수록 오차는 줄어듭니다. 학습 곡선을 세 부분으로 나누어 보면 다음과 같습니다.

- 첫 번째 부분(에포크 1~10): 학습 오차와 검증 오차가 급격히 줄어듭니다.
- 두 번째 부분(에포크 11~30): 오차의 감소세가 꺾입니다.
- 세 번째 부분(에포크 31~100): 오차가 완만하게 줄어듭니다.

세부적으로 살펴보면 학습 오차(train_err)는 약간 들쭉날쭉합니다. 그래서 학습률을 조금 작게 조정해 볼 수 있습니다. 하지만 전체적으로 오차는 감소하다 완만해지는 흐름이므로 학습률 조정은 큰 의미가 없을 것 같습니다.

모델 검증 평가

모델을 학습한 후에는 얼마나 정확하게 예측하는지 평가하는 과정이 필요합니다. 앞에서 살펴보았듯이 다중 분류 모델은 여러 개의 범주를 예측하므로 이진 분류와는 조금 다른 방식으로 평가해야 합니다.

다중 분류 모델로 예측하고 이후 처리되는 작업을 단계별로 설명하면 다음과 같습니다.

1. 모델로 예측하기
2. 소프트맥스 함수를 이용해 확률값으로 변환하기
3. 확률이 가장 큰 값의 인덱스를 추출해 해당 범주로 분류하기

절차대로 살펴보겠습니다. 모델로 예측하면 결과가 행렬(2차원 표) 형태로 나오는데, 이때 행은 데이터 개수를, 열은 분류해야 할 범주의 개수를 의미합니다. 예를 들어 데이터가 100건이고 분류해야 할 범주가 3개라면 예측 결과는 100행 3열짜리 표가 됩니다. 각각의 행은 데이터 하나의 예측 결과이고, 그 안의 3개 숫자는 각각 출력층의 로짓값을 나타냅니다.

다음 코드를 작성합니다.

CODE

```
_, pred = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)
print(pred.numpy()[:3])
```

OUTPUT

```
[[ 6.7531557  1.6497982 -8.050492 ]
 [-4.696289   0.8361721 -1.4306617]
 [-4.7747574  0.6079354 -1.373682  ]]
```

코드 실행 결과를 보면 하나의 행마다 세 개의 결괏값이 나옵니다. 각각의 값은 세 품종에 대한 입력 데이터 한 건의 분류 예측값입니다. 붓꽃 데이터 하나의 행은 Sepal.Length(꽃잎 길이), Sepal.Width(꽃잎 폭), Petal.Length(꽃받침 길이), Petal.Width(꽃받침 폭) 정보로 구성됩니다. 이 4개의 데이터를 이용해 품종을 예측하며 코드의 결과는 세 품종(setosa, versicolor, virginica)에 대한 각 노드의 예측값입니다.

아직 이 값을 봐서는 무슨 의미인지 파악하기 어렵습니다. 그러나 이 값을 소프트맥스 함수에 통과시키면 범주별 확률값으로 변환됩니다.

CODE

```
pred = nn.functional.softmax(pred, dim=1)
print(pred[:3])
```

OUTPUT

```
tensor([[9.9396e-01, 6.0396e-03, 3.7002e-07],
        [3.5719e-03, 9.0286e-01, 9.3572e-02],
        [4.0225e-03, 8.7532e-01, 1.2066e-01]])
```

코드를 실행하면 행별로 setosa, versicolor, virginica 각 품종의 확률값을 출력합니다. 첫째 행에서는 인덱스 0의 확률값이 가장 크고, 둘째, 셋째 행에서는 모두 인덱스 1의 확률값이 가장 큼니다. 풀어서 설명하면 setosa, versicolor, virginica 각각의 인덱스가 0, 1, 2 이므로 예측값의 첫째 행([9.9396e - 01, 6.0396e - 03, 3.7002e - 07])에서는 셋 중 가장 확률이 높은 setosa를 예측하고, 둘째 행([3.5719e

- 03, 9.0286e - 01, 9.3572e - 02])은 versicolor를 예측한다는 뜻입니다.

이처럼 행마다 확률값이 가장 높은 인덱스를 뽑는 작업이 필요합니다. 이를 손쉽게 처리해 주는 함수가 바로 `argmax`입니다. 이 함수를 이용해 예측값(pred)에서 행별로 가장 큰 확률을 나타내는 범주가 무엇인지 찾습니다.

CODE

```
pred = np.argmax(pred.numpy(), axis = 1) ①  
pred[:3]
```

OUTPUT

```
array([0, 1, 1])
```

① 넘파이 배열로 변환하고(`pred.numpy()`), 열 방향(`axis=1`)으로 가장 큰 값의 인덱스를 추출합니다. 행별로 가장 큰 값의 인덱스만 `pred`에 저장됩니다.

다중 분류 모델의 예측 결과는 이와 같이 `argmax` 함수를 이용해 가장 큰 값의 인덱스를 저장하는 후속 처리가 필요합니다.

이어서 모델 평가를 수행합니다.

CODE

```
print(confusion_matrix(y_val_ts.numpy(), pred))  
print(classification_report(y_val_ts.numpy(), pred, target_names=le.classes_) ①
```

OUTPUT

```
[[13  0  0]  
 [ 0 18  0]  
 [ 0  4 10]]
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	13
versicolor	0.82	1.00	0.90	18
virginica	1.00	0.71	0.83	14
accuracy			0.91	45
macro avg	0.94	0.90	0.91	45
weighted avg	0.93	0.91	0.91	45

① `target_names=le.classes_`: `target_names` 옵션을 지정하지 않으면 현재 저장된 데이터로 결과가 나옵니다. 즉, `setosa`, `versicolor`, `virginica`가 아닌 인덱스 0, 1, 2로 표시됩니다. 그러나 0, 1, 2로 표시되면 각각의 정수가 어떤 범주를 의미하는지 이해하기 어려우므로 원 범주의 이름을 표시하기 위해 `target_names` 옵션을 사용합니다. `le.classes_`에는 정수를 인코딩하기 전 범주의 이름이 저장되어 있습니다.

`classification_report` 함수의 결과는 검증 데이터 세트의 예측값과 목표값을 비교해서 평가한 내용입니다. 함수의 실행 결과 중 `support`라는 항목이 있는데, 이는

데이터의 건수입니다. 검증 데이터 세트는 총 45건이고 *setosa*, *versicolor*, *virginica*가 각각 13, 18, 14건입니다.

성능 평가 결과를 살펴보겠습니다. 먼저 정분류율(Accuracy)이 0.91입니다. 총 45건의 데이터 가운데 각각의 범주를 맞춘 비율이 0.91이라는 의미입니다. 다중 분류일 경우에는 특히 범주별 성능을 살펴보는 일이 중요합니다. 분류 보고서의 결과와 혼동 행렬(Confusion Matrix)을 함께 보며 설명하겠습니다.

*setosa*는 정밀도(Precision)와 재현율(Recall) 모두 1입니다. 1일 차에서 재현율은 목פות값을 기준으로, 정밀도는 예측값을 기준으로 얼마나 맞췄는지를 집계하는 평가 도구라고 설명했습니다. 정밀도가 1이라는 의미는 13번 예측해서 *setosa* 13개의 데이터를 모두 맞춘 것입니다. 또한 재현율이 1이라는 의미는 실제로 *setosa* 13개를 모두 맞췄다는 의미입니다.

[표 5-3] 혼동 행렬 결과로부터 *setosa*의 정밀도, 재현율 계산

구분		예측값			
		setosa	versicolor	virginica	
목פות값	setosa	13	0	0	재현율 = 13/(13+0+0)
	versicolor	0	18	0	
	virginica	0	4	10	

$$\text{정밀도} = 13/(13+0+0)$$

*versicolor*는 정밀도가 0.82, 재현율은 1입니다. [표 5-4]를 보면 *versicolor*의 실제 데이터 18개(0+18+0) 중에서 *versicolor*로 예측한 값이 18개이므로 재현율이 1입니다. 정밀도는 *versicolor*로 예측한 값(0+18+4) 중에서 실제 *versicolor*가 18개이므로 $18/22 = 0.82$ 가 됩니다.

[표 5-4] 혼동 행렬 결과로부터 *versicolor*의 정밀도, 재현율 계산

구분		예측값			
		setosa	versicolor	virginica	
목פות값	setosa	13	0	0	재현율 = 18/(0+18+0)
	versicolor	0	18	0	
	virginica	0	4	10	

$$\text{정밀도} = 18/(0+18+4)$$

마지막으로 virginica의 정밀도는 1, 재현율은 0.71입니다. 10번 예측해서 10번을 다 맞췄지만, virginica 14건 중 10건만 맞췄으므로 재현율은 0.71입니다.

[표 5-5] 혼동 행렬 결과에서 virginica의 정밀도, 재현율 계산

구분		예측값			
		setosa	versicolor	virginica	
목표값	setosa	13	0	0	재현율 = 10/(0+4+10)
	versicolor	0	18	0	
	virginica	0	4	10	

정밀도 = 18/(0+0+10)

여러분의 혼동 행렬 결과가 책과 다를 수 있습니다. 이는 딥러닝을 학습할 때 가중치의 초깃값이 랜덤하게 지정되면서 학습이 진행되기 때문입니다.

지금까지 iris 데이터를 사용해 간단한 다중 분류 모델을 구성했습니다. 다음에는 데이터도 복잡하고 모델도 은닉층을 추가한 심화 모델을 만듭니다.

다중 분류 모델링 실습 2 - MNIST

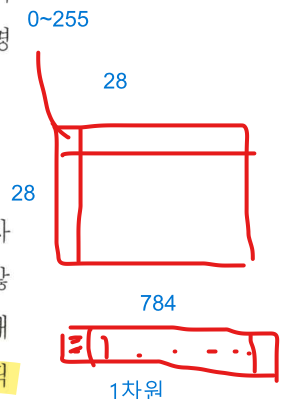
이번에는 매우 유명한 데이터 세트인 MNIST를 이용해 다중 분류 모델을 학습시킵니다. 앞에서 사용한 '5일차_다중분류모델링.ipynb' 파일에서 이어 작성합니다. 이번 절에서는 반복 코드는 생략하고 새롭게 등장하는 코드 중심으로 작성하고 설명합니다.

학습 데이터

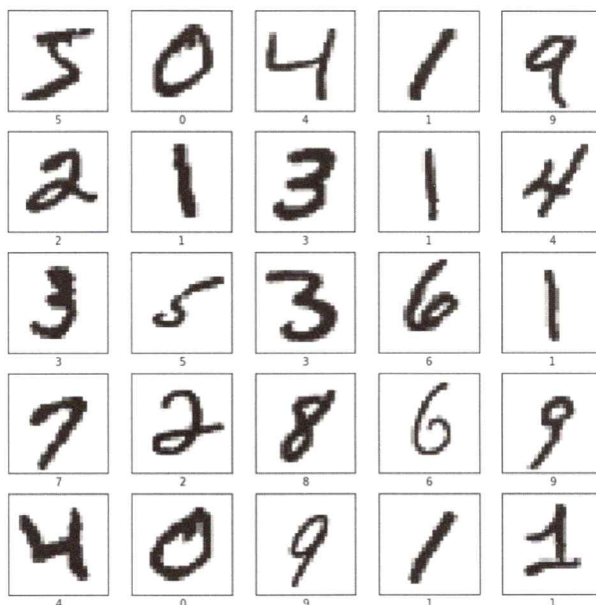
MNIST(Modified National Institute of Standards and Technology)는 손으로 쓴 숫자(0~9) 이미지 데이터 세트입니다. 딥러닝과 머신러닝을 처음 시작하는 사람들이 많이 사용하는 데이터 세트 중 하나로 다중 분류(Multi-Class Classification) 문제를 해결하는 대표적인 예제로 활용되고 있습니다. 데이터 세트는 28(세로)×28(가로) 픽셀 이미지 총 70,000개로 이루어져 있습니다(학습용: 60,000개, 테스트용: 10,000개).

이미지모델링 생성

기본적인 신경망 모델, CNN 모델, 오토인코더(Autoencoder) 모델을 학습시킬 때도 이 데이터를 많이 활용합니다. 이 데이터를 이용한 CNN 모델은 8일 차, 오토



인코더는 10일 차에서 살펴볼 예정입니다. 이번 차시에서는 MNIST 데이터로 nn.Linear 함수를 이용하는 기본 신경망 모델을 만듭니다.



[그림 5-5] MNIST 데이터 세트

데이터 다운로드 및 정보 확인하기

다중 분류 실습 1에서 작성한 라이브러리 로딩, 디바이스 준비, 사용자 정의 함수는 그대로 사용합니다. 데이터 세트를 불러오는 내용이 조금 달라집니다.

데이터 세트 다운로드

지금까지 이 책에서는 필자가 업로드한 깃허브(Github) 경로(링크)를 입력해 데이터를 다운로드했습니다. 그러나 MNIST 데이터 세트는 torchvision 라이브러리의 datasets 모듈을 이용해 다운로드합니다(깃허브는 데이터 업로드의 제한이 있습니다).

CODE

```
from torchvision import datasets ①
from torchvision.transforms import ToTensor ②
train_dataset = datasets.MNIST( ③
    root="data", ④
```

```

train=True,
download=True,      ⑥
transform=ToTensor() ⑦
)
test_dataset = datasets.MNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor() )

```

- ① torchvision의 datasets 모듈을 불러옵니다. 이 모듈은 MNIST 외에도 다양한 데이터 세트를 제공합니다.
- ② 이미지 데이터를 0~1 사이의 텐서 값으로 바꿔 주는 ToTensor 함수를 가져옵니다.
- ③ datasets.MNIST: MNIST를 다운로드합니다.
- ④ 데이터를 저장할 디렉터리를 저장합니다. 현재 위치한 코랩 폴더 아래에 'data' 폴더를 생성합니다.
- ⑤ 학습(train) 데이터를 로드(True, 만약 False면 테스트(test) 데이터를 로드)합니다.
- ⑥ 데이터를 저장할 폴더 경로를 'data'로 지정합니다. 이 폴더에 MNIST 데이터를 저장합니다. 코랩 화면에서 data 폴더를 확장하면 MNIST 폴더를 확인할 수 있습니다.
- ⑦ transform=ToTensor(): 이미지 데이터를 파이토치 텐서로 변환(스케일링된 데이터)합니다.

③~⑦의 코드는 datasets.MNIST 함수에서 사용하는 다양한 옵션입니다. 각각의 옵션은 데이터를 어디에 저장할지, 학습용 데이터를 불러올지 테스트용 데이터를 불러올지, 데이터를 어떤 형식으로 변환할지 등을 지정하는 데 사용합니다.

코드를 실행하면 다음과 같이 진행률을 표시하면서 학습용 데이터 세트 이미지(x, y)와 검증용 데이터 세트 이미지(x, y)를 각각 다운로드합니다.

```

OUTPUT
100%|██████████| 9.91M/9.91M [00:00<00:00, 16.0MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 476kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.40MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 5.43MB/s]

```

다운로드한 데이터 파일을 확인하려면 먼저 코랩 화면 왼쪽의 '파일'을 클릭해 화면을 확장한 다음, data 폴더와 MNIST를 차례로 클릭하면 됩니다. 이곳에 다운로드한 이미지들이 저장되어 있습니다.



[그림 5-6] 코랩의 파일(가상 드라이브 공간)에 저장된 MNIST 데이터 세트

데이터 세트 정보 확인하기

앞에서 저장한 학습 데이터 세트(train_dataset)의 이름을 그대로 입력하고 실행하면 데이터 세트의 정보를 확인할 수 있습니다.

CODE

```
train_dataset
```

OUTPUT

```
Dataset MNIST
  Number of datapoints: 60000
  Root location: data
  Split: Train
  StandardTransform
  Transform: ToTensor()
```

train_dataset은 텐서 데이터 세트로서 데이터 로더에 넣고 바로 사용할 수 있습니다. 이 텐서 데이터 세트는 입력 데이터(train_dataset.data)와 목표값 데이터(train_dataset.target)로 구분됩니다. 각각의 구조(shape)를 살펴보겠습니다.

CODE

```
train_dataset.data.shape, train_dataset.targets.shape
```

OUTPUT

```
(torch.Size([60000, 28, 28]), torch.Size([60000]))
```

입력 데이터의 구조는 [60000, 28, 28]입니다. 28×28 이미지가 60,000개 있다는 뜻입니다. 목표 데이터 역시 이미지 수만큼이므로 60,000개입니다.

계속해서 classes 속성으로 각 범주의 의미를 확인할 수 있습니다.

CODE

```
train_dataset.classes
```

OUTPUT

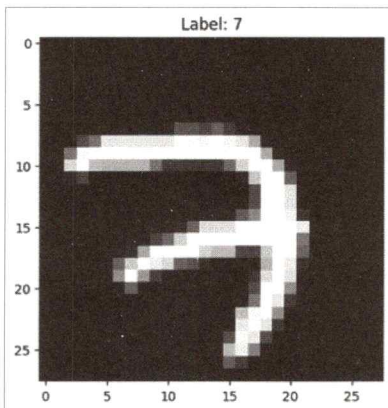
```
['0 - zero', '1 - one', '2 - two', '3 - three', '4 - four', '5 - five', '6 - six',  
'7 - seven', '8 - eight', '9 - nine']
```

0~9까지의 숫자 범주가 있음을 알 수 있습니다.

이제 데이터 세트에서 특정 이미지 데이터 하나를 추출해 확인하겠습니다.

CODE

```
n = 5000  
image, label = train_dataset.data[n], train_dataset.targets[n] ①  
  
# 이미지 시각화  
plt.imshow(image, cmap="gray") ②  
plt.title(f"Label: {label}")  
plt.show()
```

OUTPUT

- ① 인덱스 5000의 이미지와 목표값을 각각 변수 `image`, `label`에 저장합니다.
- ② `imshow` 메서드에 `image`를 넣고 흑백(`cmap="gray"`)으로 시각화합니다.

이미지를 확인하면 손글씨로 쓴 7의 이미지가 나타납니다. 사실 출력 이미지가 7인지 한글의 'ㅋ'인지 구분하기 어렵습니다. MNIST 이미지에는 사람마다 숫자를 쓰는 방식이 조금씩 다른 다양한 숫자 이미지가 모여 있습니다. `n`의 값을 바꿔 가며 다른 손글씨 이미지도 확인해 보길 바랍니다.

데이터 전처리

다운받은 데이터 세트는 x, y 분리, train과 test 분리, 그리고 스케일링을 모두 적용한 텐서 데이터입니다. 따라서 데이터 전처리를 위해서는 다음 두 가지 작업만 수행하면 됩니다.

- 학습 데이터: 학습 데이터 세트로 데이터 로더를 구성합니다.
- 검증, 테스트 데이터 세트: dataset의 data 속성으로 데이터를 추출하면 원본 데이터(스케일링이 안 된 데이터 세트)로 나옵니다. 따라서 스케일링을 수행해야 합니다.

학습 데이터

train_dataset을 데이터 로더 함수(DataLoader)에 넣고 데이터 로더를 구성합니다.

CODE

```
batch_size = 64  
train_loader = DataLoader(train_dataset, batch_size=batch_size) ①
```

① DataLoader 함수에서 다운받은 train_dataset(텐서 데이터 세트)을 입력하고 배치 크기를 64로 지정합니다. 기본값은 32이지만, 데이터 건수가 많다면 32보다 크게 정할 수도 있습니다.

검증, 테스트 데이터 세트

데이터 세트의 역할을 다시 한번 복습합니다.

- train: 학습 데이터
- validation: 검증 데이터. 학습 과정에서 에포크마다 성능을 검증하거나 모델을 생성하고 검증 데이터 세트로 한꺼번에 모델을 평가
- test: 테스트 데이터. 모델의 학습이 끝나고 실제로 운영, 배포하기 전에 최종 평가 용도로 사용. 마치 운영 과정에서 발생한 데이터처럼 간주하는 데이터 세트

검증, 테스트 데이터 세트는 가중치를 업데이트(학습)하기 위한 용도가 아니라 학습된 모델로 예측하기 위한 데이터 세트이므로 배치 단위로 데이터를 로딩할 필요가 없습니다. 따라서 다운받은 test_dataset의 data 속성으로 데이터를 직접 추출해 사용합니다. 그런데 test_dataset.data로 데이터를 추출하면 몇 가지 전처리 작

업을 수행해야 합니다. 즉, `test_dataset`을 검증과 테스트 데이터로 분할하고 스케일링한 다음, 차원을 맞추는 전처리 작업이 필요합니다.

먼저 `test_dataset.data`를 `val`과 `test`로 5,000건씩 나눕니다. 코드는 다음과 같이 인덱스를 기준으로 자르면 됩니다.

CODE

```
x_val, x_test = test_dataset.data[:5000], test_dataset.data[5000:]
y_val, y_test = test_dataset.targets[:5000], test_dataset.targets[5000:]
x_val.shape, y_val.shape
```

OUTPUT

```
(torch.Size([5000, 28, 28]), torch.Size([5000]))
```

코드를 실행해 검증, 테스트 데이터를 각각 만들었습니다. 이어서 `x_val`과 `x_test`를 스케일링합니다. 일반적으로 이미지 데이터는 0~255 사이의 값이므로 255로 나누면 자연스럽게 0~1 사이의 값(정규화)으로 스케일링됩니다.

CODE

```
x_val = x_val / 255
x_test = x_test / 255
```

스케일링까지 수행했다면 마지막으로 데이터의 차원을 맞춥니다. 이 부분은 조금 어려울 수 있으니 여기서는 대략적인 내용만 이해하고 8일 차 이미지 모델링에서 좀 더 자세히 공부하겠습니다.

이미지 하나의 데이터 크기는 [채널, 세로 픽셀 수, 가로 픽셀 수]입니다. 이미지 하나가 3차원으로 표현된다는 의미입니다. 채널은 흑백일 때 1, 컬러일 때는 3(R, G, B)입니다. MNIST 이미지 크기는 28×28입니다. 흑백 이미지이므로 크기는 [1, 28, 28]입니다.

`x_val`의 데이터 크기를 보면 [5000, 28, 28]로 3차원 데이터 세트입니다. 28×28 크기의 이미지가 5,000개 있다는 뜻입니다. 채널은 흑백이므로 1인데, 길이가 1인 차원은 생략하기도 합니다. 여기서는 원래대로 3차원 데이터 세트(데이터 건수, 세로 픽셀 수, 가로 픽셀 수)에 채널 축을 추가해 4차원(데이터 건수, 채널 수, 세로 픽셀 수, 가로 픽셀 수)으로 변환합니다.

CODE

```
x_val = x_val.view(5000, 1, 28, 28) ①
x_test = x_test.view(5000, 1, 28, 28)
```

```
print(x_val.shape, x_test.shape)
```

OUTPUT

```
torch.Size([5000, 1, 28, 28]) torch.Size([5000, 1, 28, 28])
```

- ① view 메서드는 넘파이의 reshape 속성과 동일합니다. [5000, 28, 28]의 3차원을 [5000, 1, 28, 28]의 4차원으로 변환합니다. 두 번째 축으로 흑백 채널 1개를 추가했습니다.

학습을 위한 데이터 로더와 검증 및 테스트 데이터 세트까지 모두 준비했습니다. 데이터 구조를 맞추는 게 조금 복잡해 보이나요? 지금은 이 부분을 너무 신경 쓰지 말고 그냥 넘어가도 괜찮습니다. 나중에 본격적으로 이미지 모델링을 배울 때 차근 차근 익히면 됩니다.

모델링

모델링은 모델 선언→학습→학습 곡선 확인→예측과 검증 평가순으로 진행합니다. 학습에서 에포크마다 오차를 계산할 때는 검증 데이터를 사용하고, 모델을 생성하고 최종 평가를 할 때는 테스트 데이터를 사용하겠습니다.

모델 선언

먼저 한 개의 층으로 구성된 모델 구조를 선언합니다.

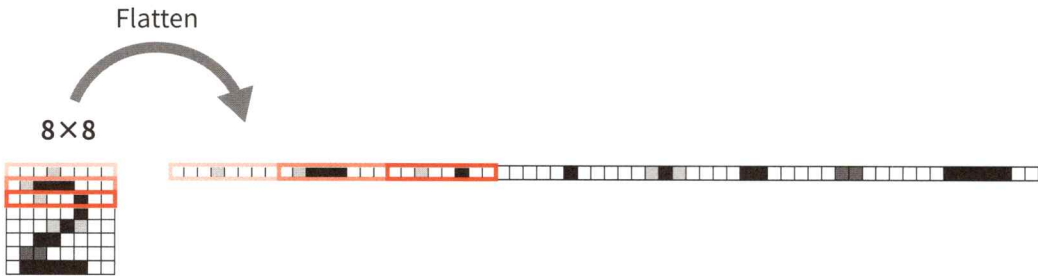
코드를 보면 한 가지 특별한 함수가 추가되어 있습니다. 바로 `nn.Flatten` 함수입니다.

CODE

```
n_feature = 28 * 28
n_class = 10
# 모델 구조 설계
model = nn.Sequential(nn.Flatten(), ①
                      nn.Linear(n_feature, n_class) ②
                      ).to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=0.001)
```

- ① `nn.Flatten` 함수는 하나의 크기가 [1, 28, 28]인 이미지를 옆으로 펼쳐 [1, 784]의 크기로 만듭니다.
- ② 다중 분류 모델의 출력층입니다. 은닉층 없이 하나의 층으로 이루어졌으며 입력 데이터를 10개의 범주로 분류해 내보냅니다.

여기서 이미지를 옆으로 펼친다(Flatten)는 말이 이해가 되나요? 조금 쉽게 설명하겠습니다. [그림 5-7]을 참고합니다.



[그림 5-7] 이미지를 펼치기(Flatten)

이미지를 펼친다는 것은 [그림 5-7]처럼 8×8 이미지가 있다면 이를 옆으로 펼쳐 1×64 크기로 만든다는 의미입니다. MNIST 데이터 세트의 각 표본은 2차원(28×28 픽셀) 혹은 3차원(1×28×28픽셀) 형태의 이미지입니다. 하지만 딥러닝 신경망에서 일반적인 층(nn.Linear)은 1차원 벡터 형태의 입력을 기대합니다. 따라서 2차원(혹은 3차원) 이미지를 1차원 벡터로 변환하는 과정이 필요하며 이를 위해 Flatten 연산을 수행합니다. 코드처럼 nn.Linear를 수행하기 전에 nn.Flatten 함수를 추가하면 이를 손쉽게 구현할 수 있습니다.

학습

학습 코드는 회귀, 이진 분류, 다중 분류 모두 동일합니다. 다만 MNIST 데이터는 에포크를 10 정도만 지정해도 충분히 학습되므로 이 부분만 잘 조절하면 준수한 성능의 모델을 학습시킬 수 있습니다.

CODE

```
epochs = 10
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model, loss_fn, optimizer, device)
    val_loss, _ = evaluate(x_val, y_val, model, loss_fn, device)
    tr_loss_list.append(tr_loss)
    val_loss_list.append(val_loss)
    print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")
```

OUTPUT

```
Epoch 1, train loss : 0.548491, val loss : 0.427864
Epoch 2, train loss : 0.328663, val loss : 0.378509
Epoch 3, train loss : 0.299855, val loss : 0.360346
```

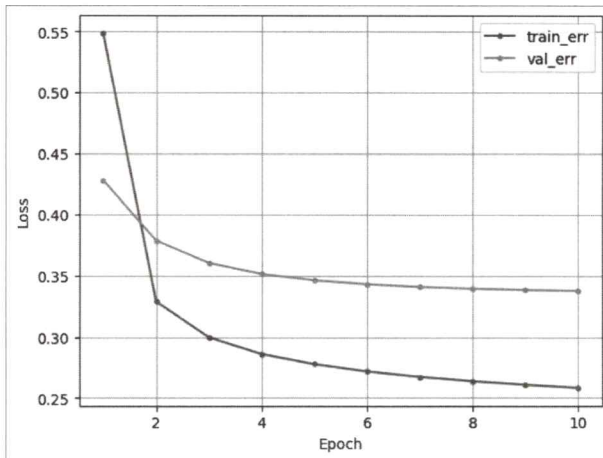
```
...
Epoch 8, train loss : 0.263588, val loss : 0.339318
Epoch 9, train loss : 0.260663, val loss : 0.338279
Epoch 10, train loss : 0.258198, val loss : 0.337550
```

이어서 학습 곡선을 그립니다.

CODE

```
dl_learning_curve(tr_loss_list, val_loss_list)
```

OUTPUT



학습 오차와 검증 오차 모두 초반에 급격히 떨어지다가 꺾인 다음, 완만한 흐름이 이어집니다.

모델 검증: 예측

학습이 모두 끝났다면 테스트 데이터로 예측하고 모델을 평가합니다.

CODE

```
_, pred = evaluate(x_test, y_test, model, loss_fn, device)
pred = nn.functional.softmax(pred, dim=1)
print(pred[:5])
```

OUTPUT

```
[[1.3157e-03, 7.6645e-06, 8.5232e-04, 6.3587e-01, 4.4115e-04, 3.5711e-01, 3.0850e-06, 1.0473e-05, 3.7079e-03, 6.7840e-04],
 [2.8627e-04, 8.3443e-08, 3.4709e-03, 8.5633e-03, 2.6991e-01, 4.1475e-03, 3.6504e-04, 3.7643e-03, 2.3072e-01, 4.7877e-01],
 [8.2669e-07, 3.1268e-11, 1.3960e-07, 1.5778e-06, 5.0594e-03, 1.3580e-06, 1.2251e-07, 8.5813e-04, 2.6261e-03, 9.9145e-01],
 [2.2787e-07, 1.3771e-09, 1.0081e-06, 5.0442e-06, 3.1541e-07, 2.7021e-04, 9.5360e-09, 2.5089e-08, 9.9927e-01, 4.5274e-04],
 [7.4606e-05, 7.1420e-12, 3.6670e-06, 1.0914e-07, 9.9845e-01, 2.4571e-05, 5.7927e-05, 3.0884e-05, 1.4698e-04, 1.2089e-03]]
```

실행 결과를 보면 10개 범주의 확률값 5건이 출력됩니다. 이 데이터를 `argmax` 함수에 적용해 값이 가장 큰 인덱스를 추출합니다. 이 값이 바로 최종 분류한 예측값입니다.

CODE

```
pred = np.argmax(pred.cpu().numpy(), axis = 1)
print(pred[:5])
```

OUTPUT

```
[3 9 9 8 4]
```

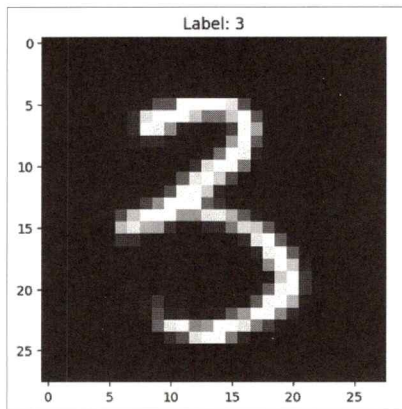
첫 번째 이미지는 3이고 두 번째 이미지는 9라는 뜻입니다. 이제 모델이 목פות값을 잘 맞추었는지 확인합니다.

다음은 앞에서 이미지를 시각화할 때 사용했던 코드입니다.

CODE

```
n = 0
image, label = test_dataset.data[5000+n], test_dataset.targets[5000+n] ①
plt.imshow(image, cmap="gray")
plt.title(f"Label: {label}")
plt.show()
```

OUTPUT



① `5000+n`: 앞에서 `test_dataset.data`를 검증 데이터(인덱스 0~4999)와 테스트 데이터로 분할했습니다. 따라서 코드를 인덱스 `5000+n`으로 지정합니다.

이 책에서는 첫 번째 이미지만 시각화했지만, 여러분은 코드에서 `n`(인덱스)을 0, 1, 2, 3, 4로 변경하면서 5개 모두 확인하기 바랍니다. 아마도 5개 모두 예측값과 목פות값이 일치할 겁니다.

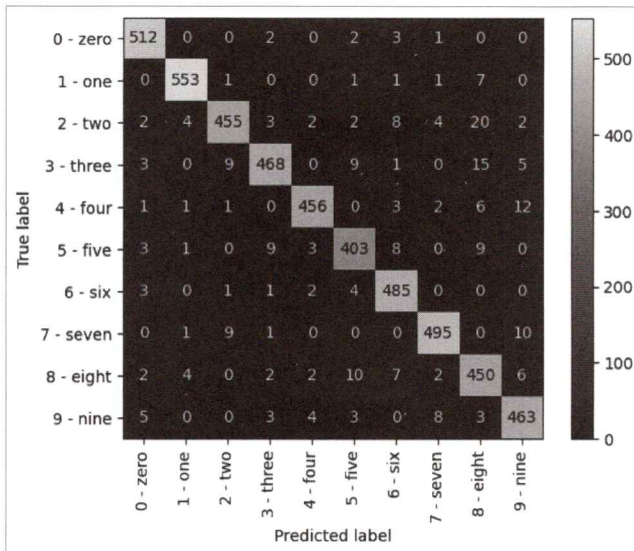
모델 검증: 평가

테스트 데이터 5,000건으로 모델을 평가합니다. MNIST 모델은 10개의 범주를 모두 살펴봐야 하므로 혼동 행렬(Confusion Matrix)을 손쉽게 이해하도록 도와주는 시각화 함수 ConfusionMatrixDisplay를 사용하겠습니다.

CODE

```
cm = confusion_matrix(y_test.numpy(), pred) ①
disp = ConfusionMatrixDisplay(cm, display_labels = train_dataset.classes) ②
disp.plot() ③
plt.xticks(rotation=90) ④
plt.show()
```

OUTPUT



- ① 혼동 행렬의 결괏값을 변수 cm에 저장합니다.
- ② ConfusionMatrixDisplay 함수의 입력으로 혼동 행렬을 저장한 변수 cm과 각 범주의 이름을 지정하고 결과를 변수 disp에 저장합니다.
- ③ 변수 disp를 plot 메서드로 시각화합니다.
- ④ x축의 예측 범주값을 90도로 돌려 표시했습니다. rotation 구문을 넣지 않으면 x축의 값들 (0-zero, 1-one, ...)이 겹쳐 표시되기 때문에 보기가 불편합니다.

혼동 행렬을 시각화하니 한눈에 알아보기 훨씬 좋지요? 범주가 많은 다중 분류 모델에서 혼동 행렬을 시각화하면 범주별로 어떤 부분이 잘못되었는지 쉽게 파악할 수 있습니다.

이제 분류 결과 보고서(classification_report)로 범주별 성능 지표를 확인합니다.

CODE

```
print(classification_report(y_test.numpy(), pred))
```

OUTPUT

	precision	recall	f1-score	support
0	0.96	0.98	0.97	520
1	0.98	0.98	0.98	564
2	0.96	0.91	0.93	502
3	0.96	0.92	0.94	510
4	0.97	0.95	0.96	482
5	0.93	0.92	0.93	436
6	0.94	0.98	0.96	496
7	0.96	0.96	0.96	516
8	0.88	0.93	0.90	485
9	0.93	0.95	0.94	489
accuracy			0.95	5000
macro avg	0.95	0.95	0.95	5000
weighted avg	0.95	0.95	0.95	5000

범주별 성능 평가 결과를 살펴보겠습니다. 정밀도(Precision)를 보면 대부분이 0.9 이상인데, 범주 8은 0.88입니다. 8로 예측했지만 맞춘 비율(Precision, 정밀도)이 다른 정밀도에 비해 가장 낮습니다. 앞에서 혼동 행렬의 시각화 결과를 보면 8로 예측했지만, 실제로는 3인 경우가 15건, 2인 경우가 20건 등 잘못 예측한 횟수가 다른 숫자에 비해 많음을 알 수 있습니다.

은닉층 추가하기

이번에는 방금 만든 모델에 은닉층 1개를 추가합니다. 방금 작성한 코드에서 모델 구조 설계 부분만 수정하면 됩니다.

CODE

```
n_feature = 28 * 28
n_class = 10
# 모델 구조 설계
model = nn.Sequential(nn.Flatten(),
                      nn.Linear(n_feature, 20), nn.ReLU(), ①
                      nn.Linear(20, n_class)                ②
                      ).to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=0.001)
```

- ① 은닉층입니다. `n_feature`를 입력으로 받아 20개(노드 수)의 정보를 출력합니다. 또한 은닉층이므로 ReLU 활성화 함수를 추가합니다.
- ② 출력층입니다. 은닉층 20개의 노드에서 내보낸 정보를 입력으로 받아 목표값 범주인 10개 노드로 예측값을 출력합니다.

모델을 설계했다면 모델 학습, 학습 곡선으로 확인, 테스트 세트로 예측하고 평가하는 순서로 코드를 작성합니다(206~209쪽 참조). 여러 번 반복했으므로 여러분이 직접 실행할 수 있을 겁니다.

여기서는 마지막 분류 모델 평가 보고서(classification_report)의 결과만 살펴 보겠습니다.

CODE

```
print(classification_report(y_test.numpy(), pred))
```

OUTPUT

	precision	recall	f1-score	support
0	0.98	0.98	0.98	520
1	0.99	0.99	0.99	564
2	0.98	0.96	0.97	502
3	0.95	0.97	0.96	510
4	0.99	0.97	0.98	482
5	0.94	0.97	0.96	436
6	0.96	0.99	0.98	496
7	0.98	0.98	0.98	516
8	0.97	0.96	0.96	485
9	0.97	0.97	0.97	489
accuracy			0.97	5000
macro avg	0.97	0.97	0.97	5000
weighted avg	0.97	0.97	0.97	5000

은닉층 없는 단순한 모델과 은닉층이 1개(20개 노드)인 모델의 성능을 비교하면 눈에 띄는 부분이 있습니다. 첫 번째 모델에서는 8의 정밀도(Precision)가 0.88이었는데, 두 번째 모델은 0.97로 꽤 많이 향상되었습니다. 정분류율 역시 0.95에서 0.97로 향상되어 모델의 전반적인 예측 성능이 개선되었습니다.

이번 차시에서는 손글씨 이미지를 옆으로 펼쳐서(Flatten) 완전 연결층으로 모델을 만들었지만, 8일 차에서는 이미지를 처리하는 알고리즘으로 모델링을 수행합니다. 이때 지금의 성능과는 또 어떻게 달라지는지 비교해 보겠습니다.

정리

이번 차시에서는 딥러닝을 활용한 다중 분류 모델링을 학습했습니다. 이제까지 배운 내용을 바탕으로 모델 구조와 사용법을 [표 5-6]으로 정리했습니다.

[표 5-6] 회귀, 이진 분류, 다중 분류 모델 구조 정리

구분	은닉층	출력층		최적화	
	활성화 함수	활성화 함수	노드 수	옵티마이저	손실 함수
회귀	ReLU	없음	1	Adam	nn.MSELoss
이진 분류	ReLU	Sigmoid	1	Adam	nn.BCELoss
다중 분류	ReLU	(소프트맥스)	범주 수	Adam	nn.CrossEntropyLoss

먼저 지금까지 다룬 모델들의 구조를 비교합니다. 회귀 모델, 이진 분류 모델, 다중 분류 모델은 모두 입력층(Input Layer), 은닉층(Hidden Layer), 출력층(Output Layer)으로 구성됩니다. 하지만 출력층의 활성화 함수(Activation Function)와 손실 함수(Loss Function)가 각각 다르다는 점이 중요한 차이입니다.

회귀 모델은 출력층에서 별도의 활성화 함수를 사용하지 않으며 예측값은 연속적인 숫자(실수, $-\infty \sim +\infty$)로 나옵니다. 손실 함수로는 주로 평균 제곱 오차(Mean Squared Error, MSE)를 사용해 모델이 예측한 값과 목표값 사이의 오차를 최소화합니다.

반면 이진 분류 모델은 특정 입력이 두 범주(예: 0 또는 1) 중 어느 쪽에 속하는지 예측합니다. 예측할 때는 출력층에 시그모이드 활성화 함수를 사용해 예측값을 0과 1 사이의 확률값으로 변환합니다. 손실 함수로는 이진 교차 엔트로피(Binary Cross Entropy, BCELoss)를 사용하며 모델이 예측한 확률값과 목표값 사이의 차이를 계산합니다.

마지막으로 다중 분류 모델은 여러 개의 범주 중에서 하나를 선택하는 문제입니다. 출력층에는 소프트맥스 활성화 함수를 사용해 모델이 각각의 범주에서 계산한 점수를 확률값으로 변환합니다. 예를 들어 MNIST 분류 문제에서는 입력 이미지가 0~9까지의 숫자 중 어떤 숫자인지를 확률로 계산하고 가장 확률이 높은 범주를 최종 예측값으로 선택합니다. 이때 손실 함수로 다중 분류 교차 엔트로피(Cross Entropy Loss)를 사용해 모델을 학습시킵니다. 다만 파이토치에서 제공하는 Cross